Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки

ЗВІТ

з лабораторної роботи №2

« Перетворення ER-діаграми на схему PostgreSQL»

з дисципліни: Бази Даних

Виконали: студенти групи ІМ-54
Дрига О.О.
Голубков В.С.
Ретізник А.Є.
Перевірив: Хмельницький А.А.

Київ - 2026

Тема: Перетворення ER-моделі на реляційну схему бази даних за допомогою SQL.
Мета: Набути практичних навичок проєктування реляційної схеми баз даних на основі існуючої ER-діаграми. Створити таблиці за допомогою SQL-інструкцій DDL у СУБД PostgreSQL, визначити типи даних, первинні та зовнішні ключі, а також застосувати необхідні обмеження (UNIQUE, NOT NULL, CHECK, DEFAULT). Заповнити таблиці тестовими даними за допомогою DML-інструкцій (INSERT) та перевірити цілісність створеної схеми.

Структура бази даних та скрипти ініціалізації (DDL та DML)

База даних складається з 10 взаємопов'язаних таблиць, що реалізують збереження даних про клієнтів, видавництва, авторів, категорії, книги (з їхніми зв'язками "багато до багатьох" через проміжні таблиці), а також замовлення, позиції замовлень та відгуки

DDL :

```
CREATE TYPE order_status AS ENUM ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled');


CREATE TABLE customer (
    customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20) UNIQUE NOT NULL,
    registration_date TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    shipping_address_default VARCHAR(255)
);


CREATE TABLE publisher (
    publisher_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name_publisher VARCHAR(100) UNIQUE NOT NULL,
    country_publisher VARCHAR(50)
);
```

```
CREATE TABLE author (
   author_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   first_name VARCHAR(100) NOT NULL,
   last_name VARCHAR(100) NOT NULL,
   country VARCHAR(50)
);

CREATE TABLE category (
   category_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   name_category VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE book (
   book_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   publisher_id INT NOT NULL REFERENCES publisher(publisher_id) ON DELETE
RESTRICT,
   isbn VARCHAR(20) UNIQUE NOT NULL,
   title VARCHAR(255) NOT NULL,
   publication_year SMALLINT NOT NULL CHECK (publication_year BETWEEN -2000 AND
2100),
   price NUMERIC(10,2) NOT NULL CHECK (price > 0),
   stock_quantity SMALLINT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0)
);

CREATE TABLE book_author (
   book_id INT NOT NULL REFERENCES book(book_id) ON DELETE CASCADE,
   author_id INT NOT NULL REFERENCES author(author_id) ON DELETE CASCADE,
```

```
    PRIMARY KEY (book_id, author_id)
);

CREATE TABLE book_category (
    book_id INT NOT NULL REFERENCES book(book_id) ON DELETE CASCADE,
    category_id INT NOT NULL REFERENCES category(category_id) ON DELETE CASCADE,
    PRIMARY KEY (book_id, category_id)
);

CREATE TABLE orders (
    order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customer(customer_id) ON DELETE RESTRICT,
    order_date TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status order_status NOT NULL DEFAULT 'Pending',
    shipping_address VARCHAR(255) NOT NULL,
    total_amount NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount >= 0)
);

CREATE TABLE order_item (
    order_item_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES book(book_id) ON DELETE RESTRICT,
    quantity SMALLINT NOT NULL CHECK (quantity > 0),
    price_at_purchase NUMERIC(10,2) NOT NULL CHECK (price_at_purchase >= 0),
    CONSTRAINT uq_order_book UNIQUE (order_id, book_id)
);
```

CREATE TABLE review (

review_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

customer_id INT NOT NULL REFERENCES customer(customer_id) ON DELETE CASCADE,

book_id INT NOT NULL REFERENCES book(book_id) ON DELETE CASCADE,

order_item_id INT REFERENCES order_item(order_item_id) ON DELETE SET NULL,

rating SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),

comment TEXT,

review_date TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT uq_customer_book_review UNIQUE (customer_id, book_id)

);

DML:

```
INSERT INTO customer (first_name, last_name, email, phone,
shipping_address_default)

VALUES

    ('Олександр', 'Шевченко', 'olexandr.sh@gmail.com', '+380671112233', 'м.
Київ, вул. Хрещатик, 15, кв. 4'),

    ('Анна', 'Коваленко', 'anna.kov@ukr.net', '+380509998877', 'м. Львів, вул.
Городоцька, 82, кв. 12'),

    ('Дмитро', 'Бондаренко', 'dmitry.bond@outlook.com', '+380934445566', 'м.
Одеса, вул. Дерибасівська, 10, кв. 2'),

    ('Марія', 'Мельник', 'm.melnyk@gmail.com', '+380687771122', 'м. Дніпро, пр.
Яворницького, 45, кв. 18');



INSERT INTO publisher (name_publisher, country_publisher)

VALUES

    ('Видавництво Старого Лева', 'Україна'),

    ('O''Reilly Media', 'США'),
```

```
    ('Penguin Random House', 'США'),
    ('Клуб Сімейного Дозвілля', 'Україна');

INSERT INTO author (first_name, last_name, country)
VALUES
    ('Роберт', 'Мартін', 'США'),
    ('Джордж', 'Орвелл', 'Велика Британія'),
    ('Сергій', 'Жадан', 'Україна'),
    ('Дж. К.', 'Роулінг', 'Велика Британія');

INSERT INTO category (name_category)
VALUES
    ('Програмування'),
    ('Фантастика та Утопія'),
    ('Сучасна проза'),
    ('Фентезі');

INSERT INTO book (publisher_id, isbn, title, publication_year, price,
stock_quantity)
VALUES
    (2, '9780132350884', 'Clean Code: A Handbook of Agile Software
Craftsmanship', 2008, 1250.00, 15),
    (3, '9780451524935', '1984', 1949, 350.00, 40),
    (1, '9789662909494', 'Інтернат', 2017, 420.00, 25),
    (4, '9786171218567', 'Гаррі Поттер і філософський камінь', 1997, 380.00,
30);
```

```
INSERT INTO book_author (book_id, author_id)
VALUES
    (1, 1),
    (2, 2),
    (3, 3),
    (4, 4);

INSERT INTO book_category (book_id, category_id)
VALUES
    (1, 1),
    (2, 2),
    (3, 3),
    (4, 4);

INSERT INTO orders (customer_id, status, shipping_address, total_amount)
VALUES
    (1, 'Delivered', 'м. Київ, вул. Хрещатик, 15, кв. 4', 1600.00),
    (2, 'Shipped', 'м. Львів, вул. Городоцька, 82, кв. 12', 420.00),
    (3, 'Processing', 'м. Одеса, вул. Дерибасівська, 10, кв. 2', 380.00),
    (1, 'Pending', 'м. Київ, вул. Хрещатик, 15, кв. 4', 350.00);

INSERT INTO order_item (order_id, book_id, quantity, price_at_purchase)
VALUES
    (1, 1, 1, 1250.00),
    (1, 2, 1, 350.00),
```

```
    (2, 3, 1, 420.00),
    (3, 4, 1, 380.00),
    (4, 2, 1, 350.00);

INSERT INTO review (customer_id, book_id, order_item_id, rating, comment)
VALUES
    (1, 1, 1, 5, 'Must-read для кожного розробника! Дуже якісна книга.'),
    (1, 2, 2, 5, 'Класика, яка залишається актуальною й сьогодні.'),
    (2, 3, 3, 4, 'Чудовий роман Жадана, дуже сильні емоції після прочитання.');
```

TEST

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

SELECT 'customer' AS tbl, COUNT(*) AS rows_count FROM customer
UNION ALL SELECT 'publisher', COUNT(*) FROM publisher UNION
ALL SELECT 'author', COUNT(*) FROM author UNION ALL SELECT
'category', COUNT(*) FROM category UNION ALL SELECT 'book',
COUNT(*) FROM book UNION ALL SELECT 'bookauthor', COUNT(*)
FROM bookauthor UNION ALL SELECT 'bookcategory', COUNT(*) FROM
bookcategory UNION ALL SELECT 'orders', COUNT(*) FROM orders
```

```
UNION ALL SELECT 'orderitem', COUNT(*) FROM orderitem UNION

ALL SELECT 'review', COUNT(*) FROM review;

SELECT b.title, p.namepublisher,

 a.firstname || ' ' || a.lastname AS author,

c.namecategory

FROM book b

JOIN publisher p ON p.publisherid = b.publisherid

JOIN bookauthor ba ON ba.bookid = b.bookid JOIN

author a ON a.authorid = ba.authorid JOIN

bookcategory bc ON bc.bookid = b.bookid JOIN

category c ON c.categoryid = bc.categoryid;


SELECT o.orderid, c.lastname, o.status, b.title,

oi.quantity, oi.priceatpurchase

FROM orders o

JOIN customer c ON c.customerid = o.customerid

JOIN orderitem oi ON oi.orderid = o.orderid JOIN

book b ON b.bookid = oi.bookid ORDER BY

o.orderid;


SELECT c.lastname, b.title, r.rating, r.comment

FROM review r

JOIN customer c ON c.customerid = r.customerid

JOIN orderitem oi ON oi.orderitemid =

r.orderitemid JOIN book b ON b.bookid = oi.bookid;
```

```
SELECT o.orderid, o.totalamount,
 SUM(oi.quantity * oi.priceatpurchase) AS calc FROM

orders o

JOIN orderitem oi ON oi.orderid = o.orderid

GROUP BY o.orderid, o.totalamount

ORDER BY o.orderid;




BEGIN



 BEGIN

 INSERT INTO customer (firstname, lastname, email)  VALUES

('Тест', 'Дубль', 'ivan.franko@example.com');  RAISE NOTICE

'C1 ПРОВАЛ: дубль Email вставився';  EXCEPTION WHEN OTHERS

THEN

 RAISE NOTICE 'C1 OK (дубль Email): [%] %', SQLSTATE, SQLERRM;  END;




 BEGIN

 INSERT INTO book (publisherid, isbn, title, price,
stockquantity)

 VALUES (1, '000-0-00-000000-0', 'Тест', 100, -5);  RAISE NOTICE 'C2

ПРОВАЛ: від''ємний StockQuantity вставився';  EXCEPTION WHEN OTHERS

THEN

 RAISE NOTICE 'C2 OK (StockQuantity = -5): [%] %', SQLSTATE,
SQLERRM;
```

```
 END;



 BEGIN

 INSERT INTO review (customerid, orderitemid, rating) VALUES (1,
NULL, 6);

 RAISE NOTICE 'С3 ПРОВАЛ: Rating = 6 вставився';

EXCEPTION WHEN OTHERS THEN

 RAISE NOTICE 'С3 ОК (Rating = 6): [%] %', SQLSTATE, SQLERRM;  END;




 BEGIN

 INSERT INTO review (customerid, orderitemid, rating) VALUES (1,
NULL, 0);

 RAISE NOTICE 'С4 ПРОВАЛ: Rating = 0 вставився';

EXCEPTION WHEN OTHERS THEN

 RAISE NOTICE 'С4 ОК (Rating = 0): [%] %', SQLSTATE, SQLERRM;  END;




 BEGIN

 INSERT INTO orders (customerid, status, shippingaddress,
totalamount)

 VALUES (999, 'Тест', 'Київ', 100);

 RAISE NOTICE 'С5 ПРОВАЛ: замовлення неіснуючого клієнта
вставилось';
 EXCEPTION WHEN OTHERS THEN

 RAISE NOTICE 'С5 ОК (customerid = 999): [%] %', SQLSTATE,
SQLERRM;
```

```
 END;



 BEGIN

 INSERT INTO review (customerid, orderitemid, rating) VALUES (1, 1,
3);

 RAISE NOTICE 'C6 ПРОВАЛ: другий відгук на позицію вставився';

EXCEPTION WHEN OTHERS THEN

 RAISE NOTICE 'C6 OK (другий відгук на orderitemid = 1): [%] %',
SQLSTATE, SQLERRM;

 END;




 BEGIN

 INSERT INTO orderitem (orderid, bookid, quantity,
priceatpurchase)

 VALUES (1, 1, 0, 300);

 RAISE NOTICE 'C7 ПРОВАЛ: Quantity = 0 вставився';

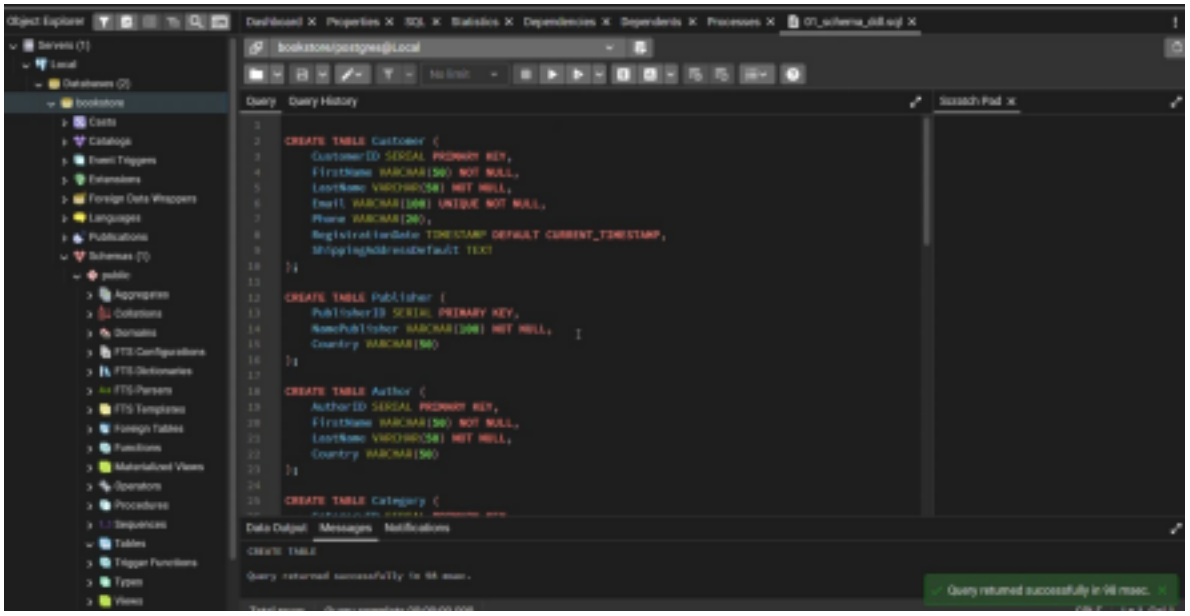
EXCEPTION WHEN OTHERS THEN

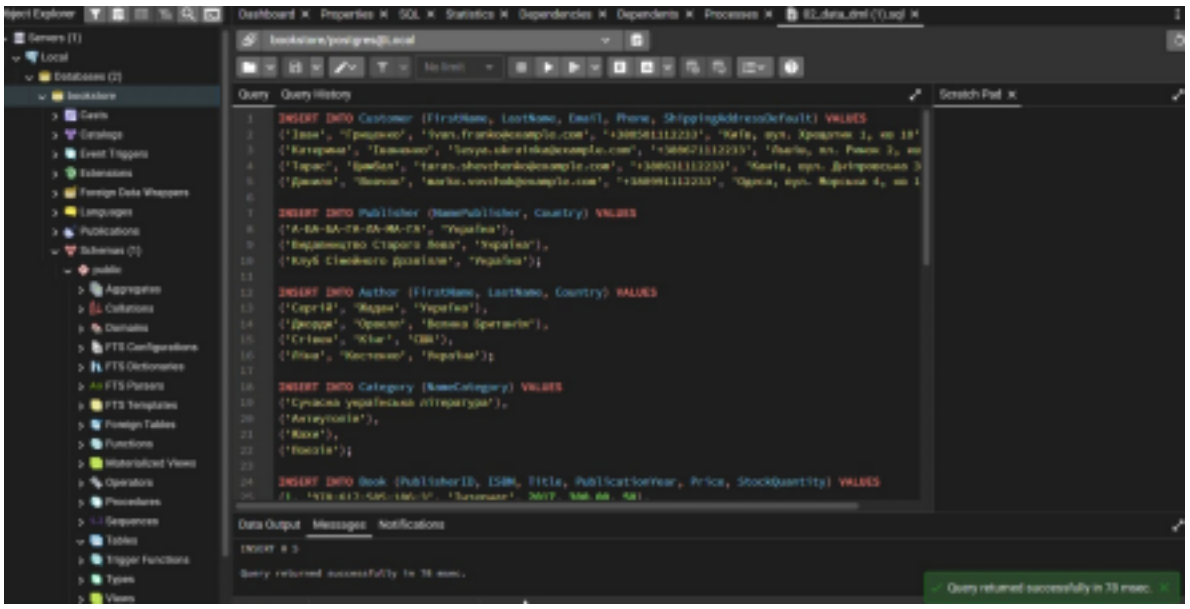
 RAISE NOTICE 'C7 OK (Quantity = 0): [%] %', SQLSTATE,
SQLERRM;

 END;

END
```

Успішне виконання DDL та DML запитів у pgAdmin:

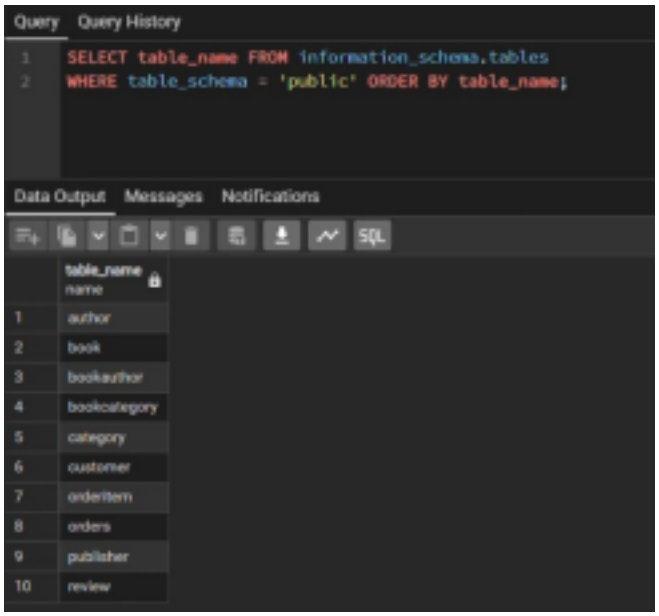


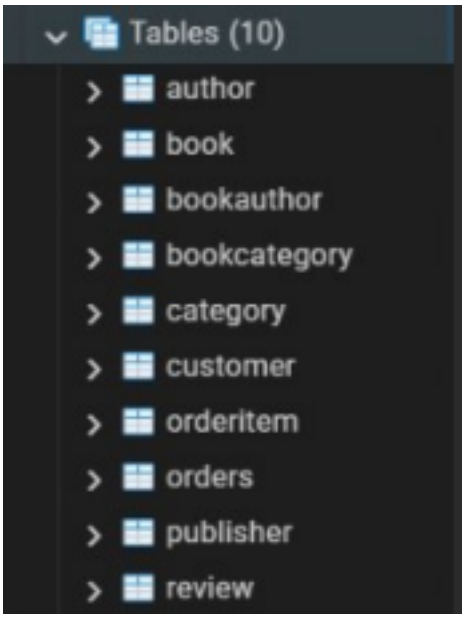


Перевірка структури та цілісності даних:

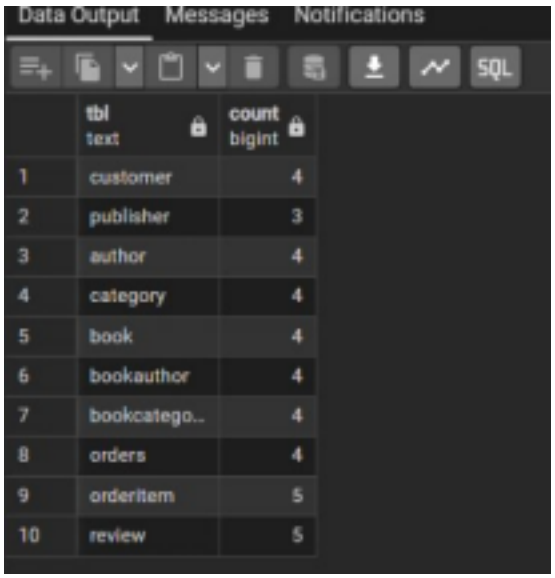
Після виконання скриптів було проведено верифікацію наповнення бази даних:

За допомогою запиту до information_schema.tables підтверджено наявність усіх 10 створених таблиць у схемі public. Перевірка наявності таблиць у базі даних:





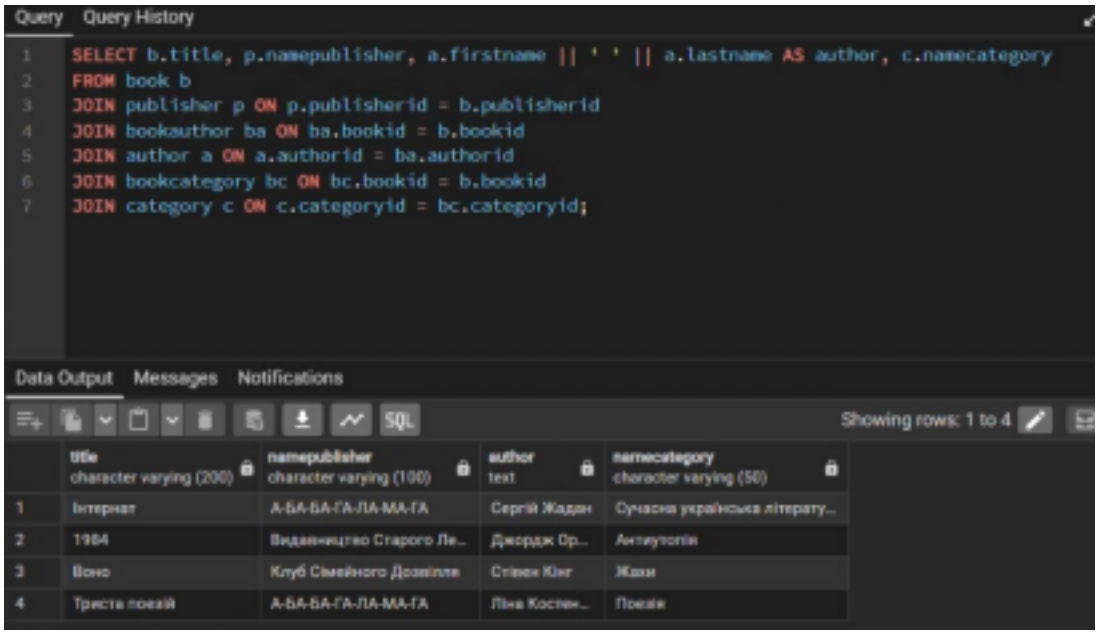
Перевірка кількості рядків за допомогою конструкції UNION ALL підтвердила очікуваний обсяг даних: 4 клієнти, 3 видавництва, 4 автори, 4 категорії, 4 книги, 4 записи у зв'язках авторів та категорій, 4 замовлення, 5 позицій замовлень та 5 відгуків.



Аналітичні запити (JOIN)

Для перевірки реляційних зв'язків виконано вибірки даних із застосуванням JOIN:

Книги з деталями: Зведено інформацію про назви книг, їхніх видавців, авторів та категорії. Вибірка книг з інформацією про авторів, видавців та категорії:



```
SELECT b.title, p.namepublisher, a.firstname || ' ' || a.lastname AS author, c.namecategory
FROM book b
JOIN publisher p ON p.publisherid = b.publisherid
JOIN bookauthor ba ON ba.bookid = b.bookid
JOIN author a ON a.authorid = ba.authorid
JOIN bookcategory bc ON bc.bookid = b.bookid
JOIN category c ON c.categoryid = bc.categoryid;
```

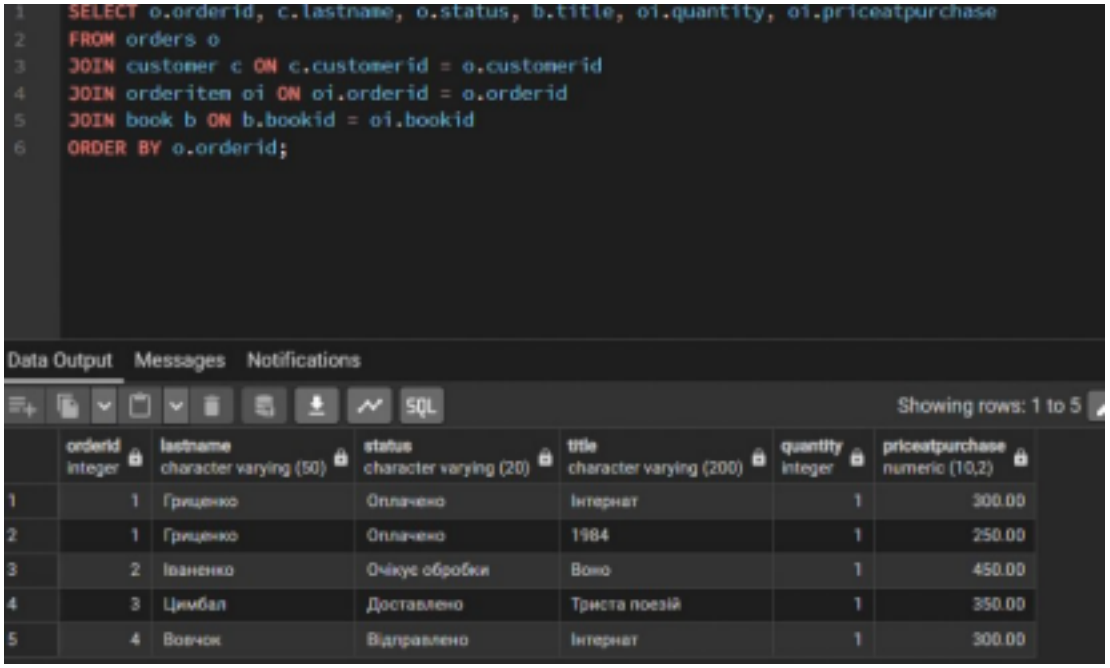
Query | Query History

Data Output | Messages | Notifications

Showing rows: 1 to 4

	title character varying (200)	namepublisher character varying (100)	author text	namecategory character varying (50)
1	Інтернат	А-БА-БА-ГА-ЛА-МА-ГА	Сергій Жадан	Сучасна українська літерату...
2	1984	Видавництво Старого Ле...	Джордж Ор...	Антиутопія
3	Воно	Клуб Сімейного Дозвілля	Стівен Кінг	Жахи
4	Триста поезій	А-БА-БА-ГА-ЛА-МА-ГА	Ліна Костен...	Поезія

Деталі замовлень: Виведено замовлення клієнтів із прив'язкою до придбаних позицій, кількості та ціни на момент покупки. Вибірка замовлень клієнтів із товарними позиціями:



```
SELECT o.orderid, c.lastname, o.status, b.title, oi.quantity, oi.priceatpurchase
FROM orders o
JOIN customer c ON c.customerid = o.customerid
JOIN orderitem oi ON oi.orderid = o.orderid
JOIN book b ON b.bookid = oi.bookid
ORDER BY o.orderid;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	orderid integer	lastname character varying (50)	status character varying (20)	title character varying (200)	quantity integer	priceatpurchase numeric (10,2)
1	1	Гриценко	Оплачено	Інтернат	1	300.00
2	1	Гриценко	Оплачено	1984	1	250.00
3	2	Іваненко	Очікує обробки	Воно	1	450.00
4	3	Цимбал	Доставлено	Триста поезій	1	350.00
5	4	Вовчок	Відправлено	Інтернат	1	300.00

Відгуки: Отримано список відгуків з інформацією про клієнта, книгу, оцінку та текст коментаря.

Валідація сум: Виконано запит, який згрупував замовлення та математично підтвердив, що загальна сума замовлення (TotalAmount) коректно збігається із сумою вартості окремих позицій (SUM(Quantity * PriceAtPurchase)).

Вибірка відгуків та математична перевірка суми чека :

```sql
SELECT o.orderid, c.lastname, o.status, b.title, oi.quantity, oi.priceatpurchase
FROM orders o
JOIN customer c ON c.customerid = o.customerid
JOIN orderitem oi ON oi.orderid = o.orderid
JOIN book b ON b.bookid = oi.bookid
ORDER BY o.orderid;
```

Data Output | Messages | Notifications

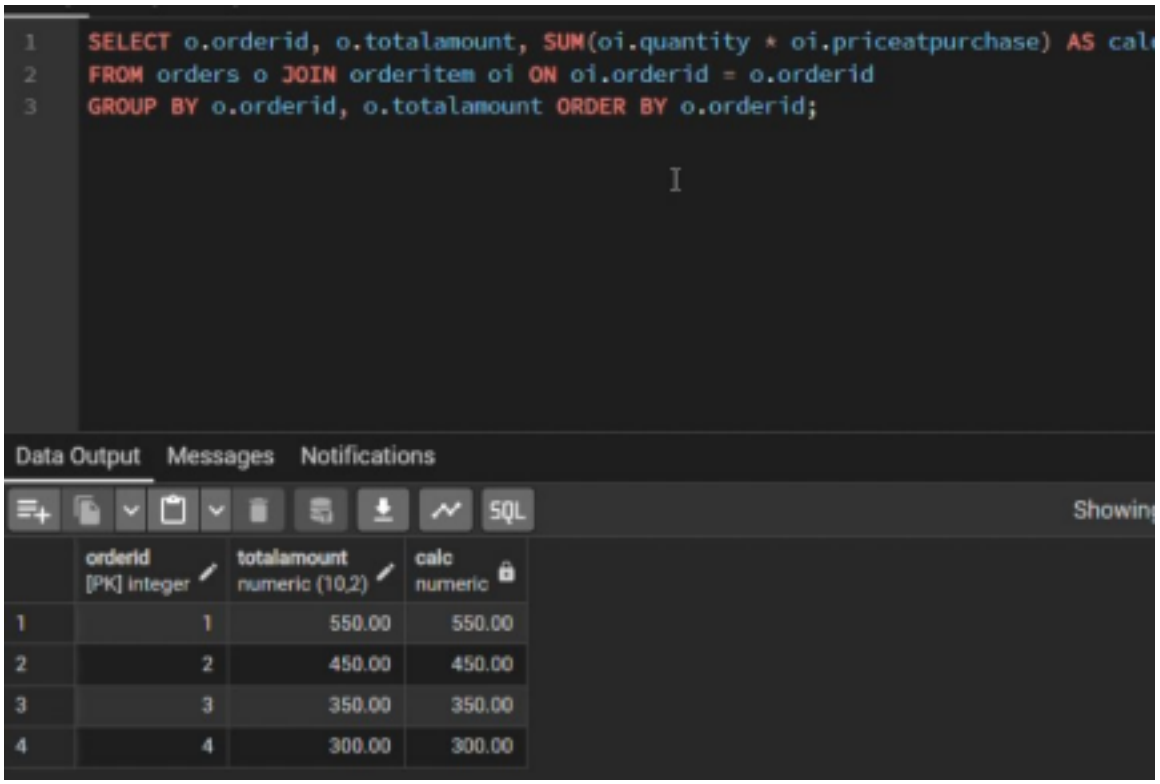
Showing rows: 1 to 5

	orderid integer	lastname character varying (50)	status character varying (20)	title character varying (200)	quantity integer	priceatpurchase numeric (10,2)
1	1	Гриценко	Оплачено	Інтернат	1	300.00
2	1	Гриценко	Оплачено	1984	1	250.00
3	2	Іваненко	Очікує обробки	Воно	1	450.00
4	3	Цимбал	Доставлено	Триста поезій	1	350.00
5	4	Вовчок	Відправлено	Інтернат	1	300.00

```sql
SELECT c.lastname, b.title, r.rating, r.comment
FROM review r
JOIN customer c ON c.customerid = r.customerid
JOIN orderitem oi ON oi.orderitemid = r.orderitemid
JOIN book b ON b.bookid = oi.bookid;
```

Data Output | Messages | Notifications

	lastname character varying (50)	title character varying (200)	rating integer	comment text
1	Гриценко	Інтернат	5	Дуже сильна книга, рекомендую!
2	Гриценко	1984	4	Класика, але трохи важко читаєтьс...
3	Іваненко	Воно	5	Страшно цікаво!
4	Цимбал	Триста поезій	5	Неймовірні вірші.
5	Вовчок	Інтернат	5	Швидка доставка, якість друку суп...

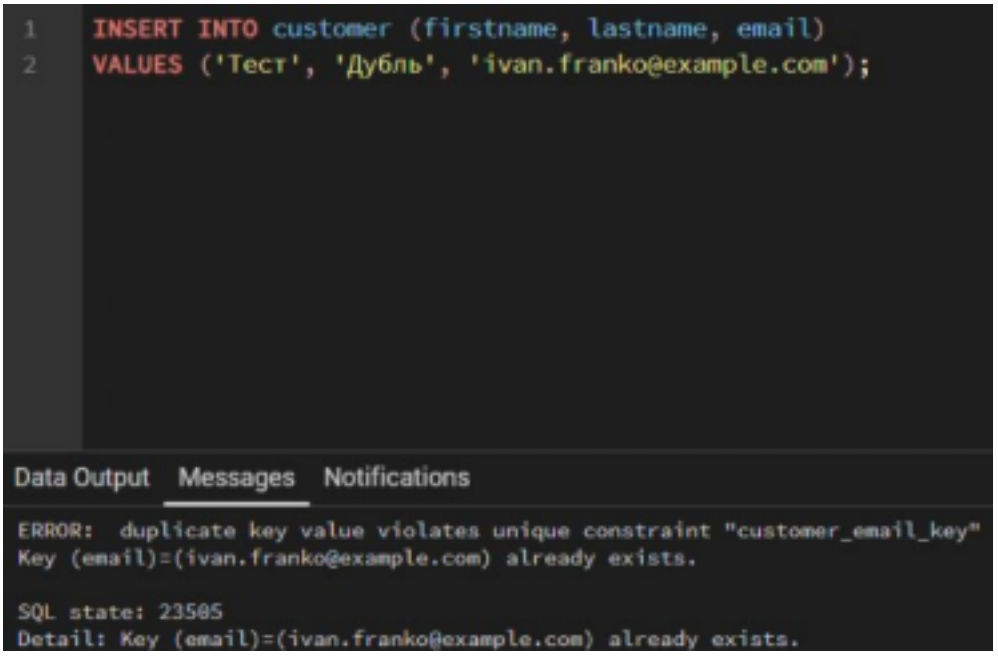


Негативне тестування обмежень (Constraints)

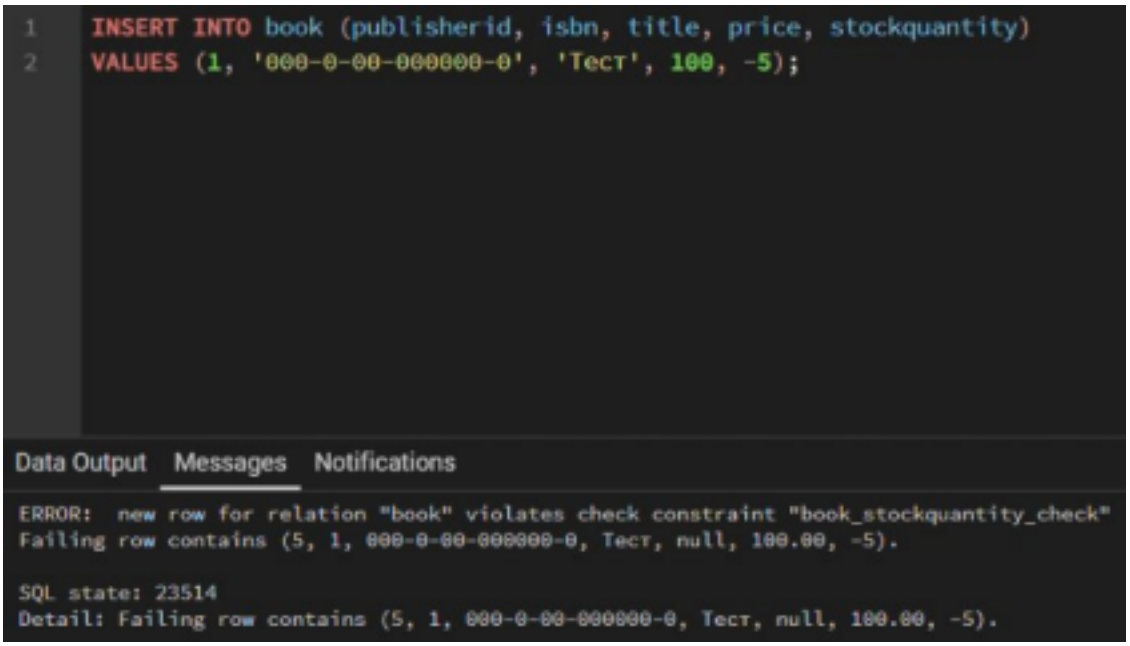
Для гарантування цілісності бази даних успішно проведено ряд негативних тестів, які підтвердили спрацювання обмежень:

- Унікальність Email (UNIQUE): Спроба додати користувача з існуючим email (ivan.franko@example.com) викликала помилку customer_email_key.
- Від'ємний залишок (CHECK): Спроба вказати кількість книг на складі -5 заблокована обмеженням book_stockquantity_check.

Спрацювання обмежень UNIQUE для email та CHECK для залишку на складі

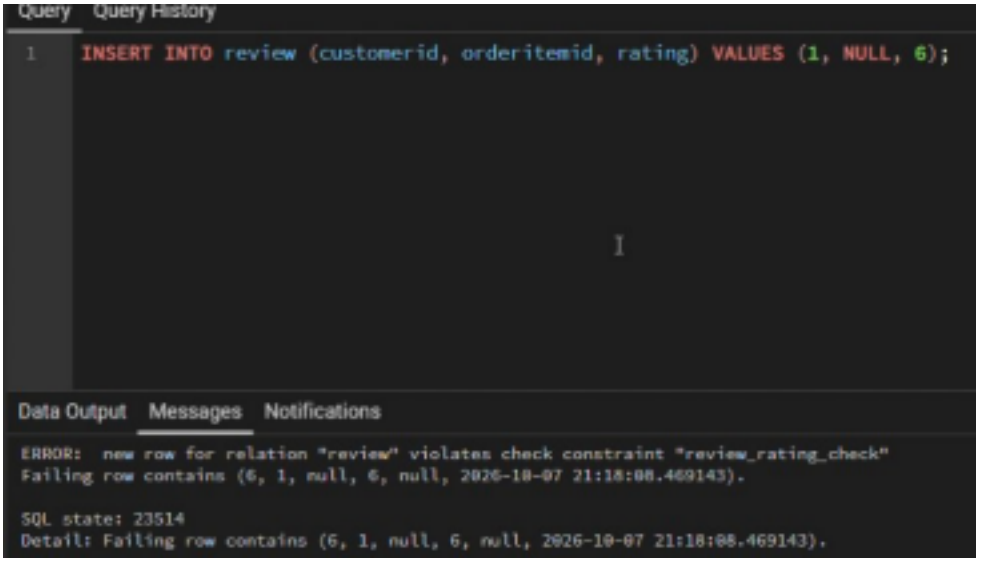


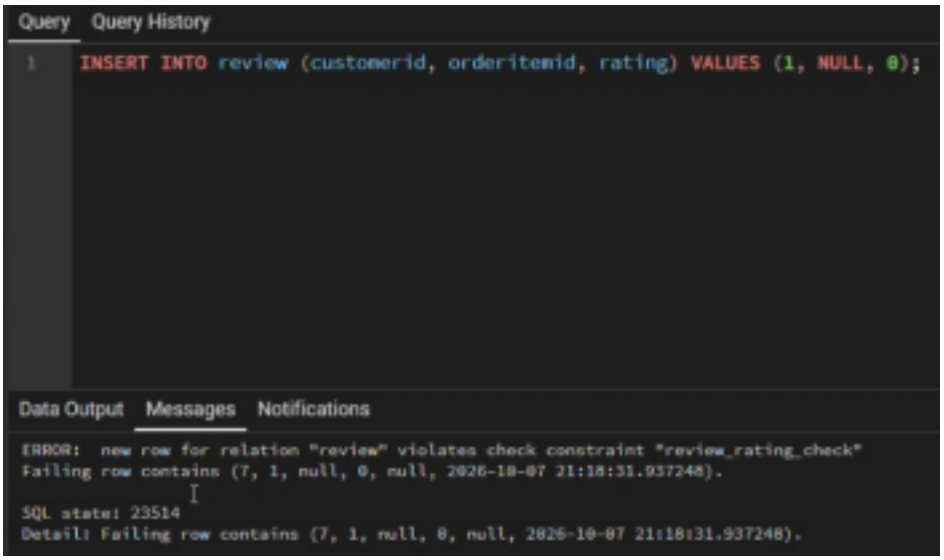
:



Некоректний рейтинг (CHECK): Вставка відгуків з оцінкою 6 та 0 відхилена через спрацювання review_rating_check.

Спрацювання обмеження CHECK для рейтингу відгуків:





Порушення зовнішнього ключа (FOREIGN KEY): Спроба створити замовлення для неіснуючого клієнта (ID=999) видала помилку orders_customerid_fkey.

Дублювання відгуків (UNIQUE): Спроба залишити другий відгук на ту саму позицію замовлення (ID=1) була заблокована review_orderitemid_key.

Порушення цілісності зовнішнього ключа та дублювання відгуку:

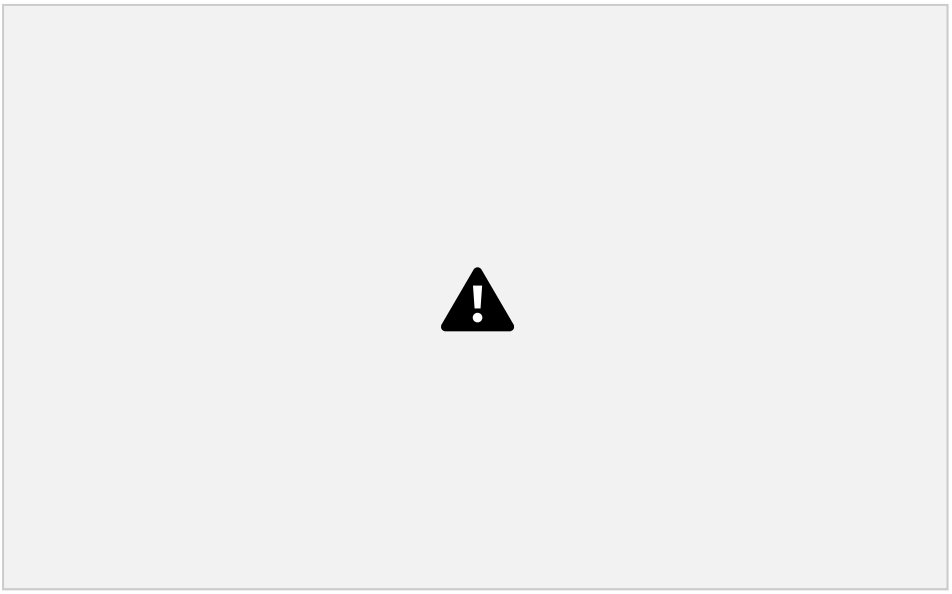
Query Query History

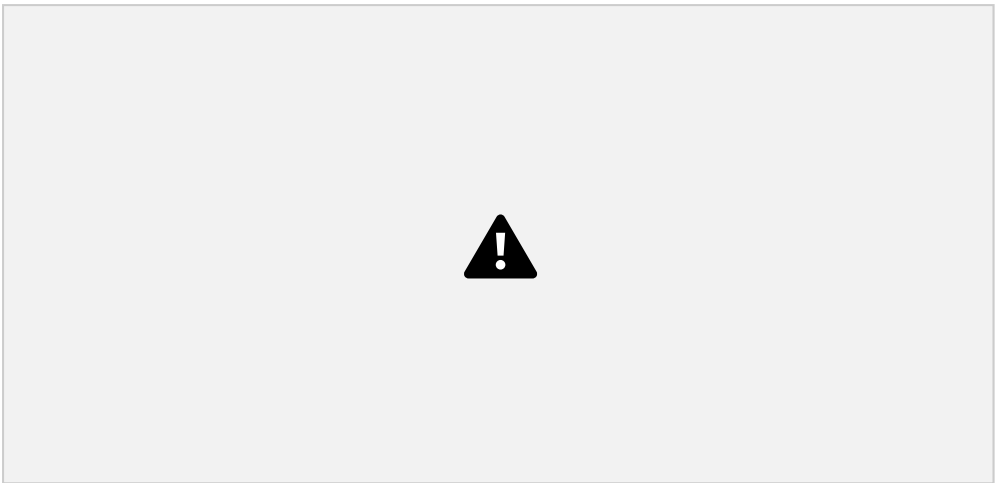
```
1  INSERT INTO orders (customerid, status, shippingaddress, totalamount)
2  VALUES (999, 'Тест', 'Київ', 100);
```

Data Output Messages Notifications

```
ERROR:  insert or update on table "orders" violates foreign key constraint "orders_customerid_fkey"
Key (customerid)=(999) is not present in table "customer".

SQL state: 23503
Detail: Key (customerid)=(999) is not present in table "customer".
```



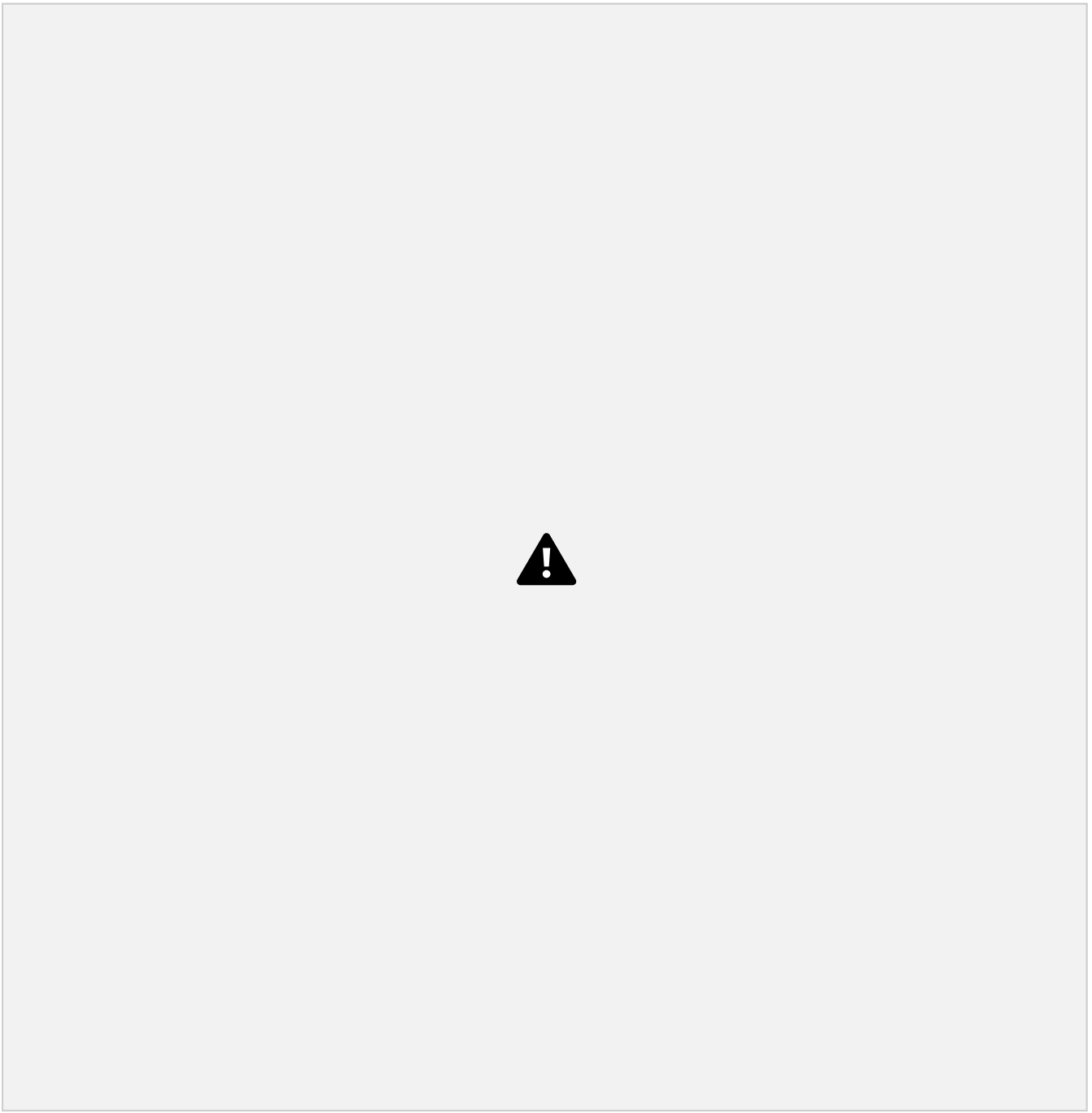


Нульова кількість у замовленні (CHECK): Додавання позиції замовлення з Quantity =

0 відхилено обмеженням orderitem_quantity_check.

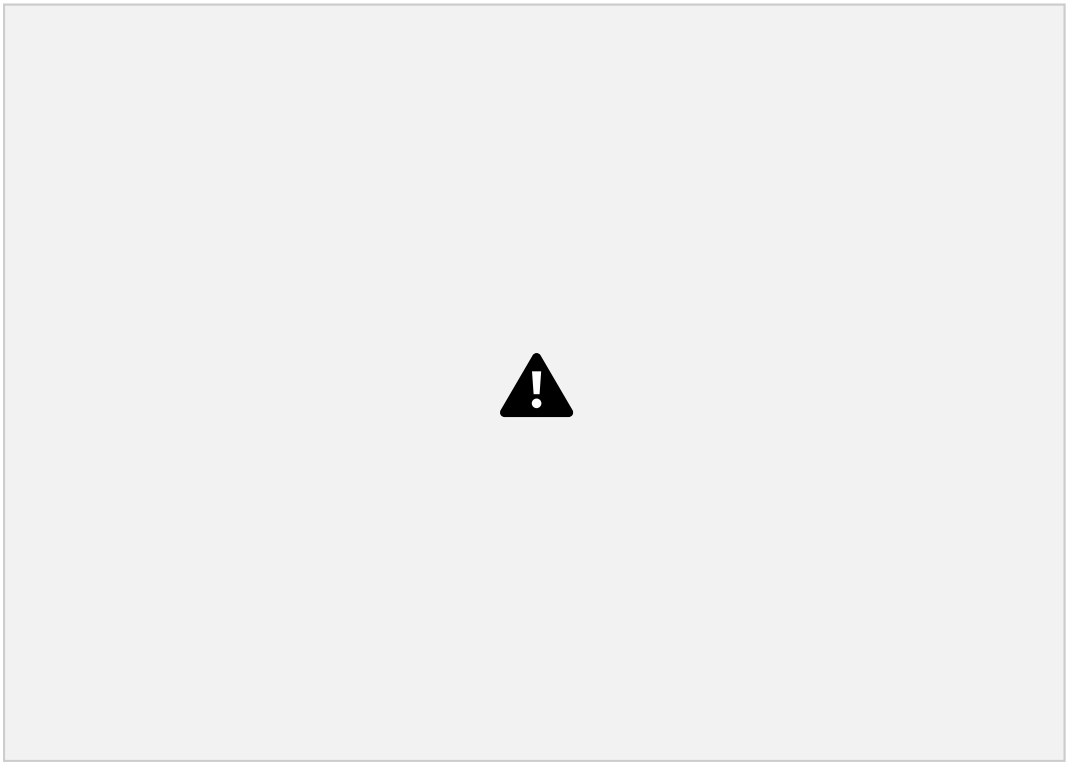
Фінальна перевірка: Повторний підрахунок рядків підтвердив, що жоден з некоректних записів не потрапив до бази даних.

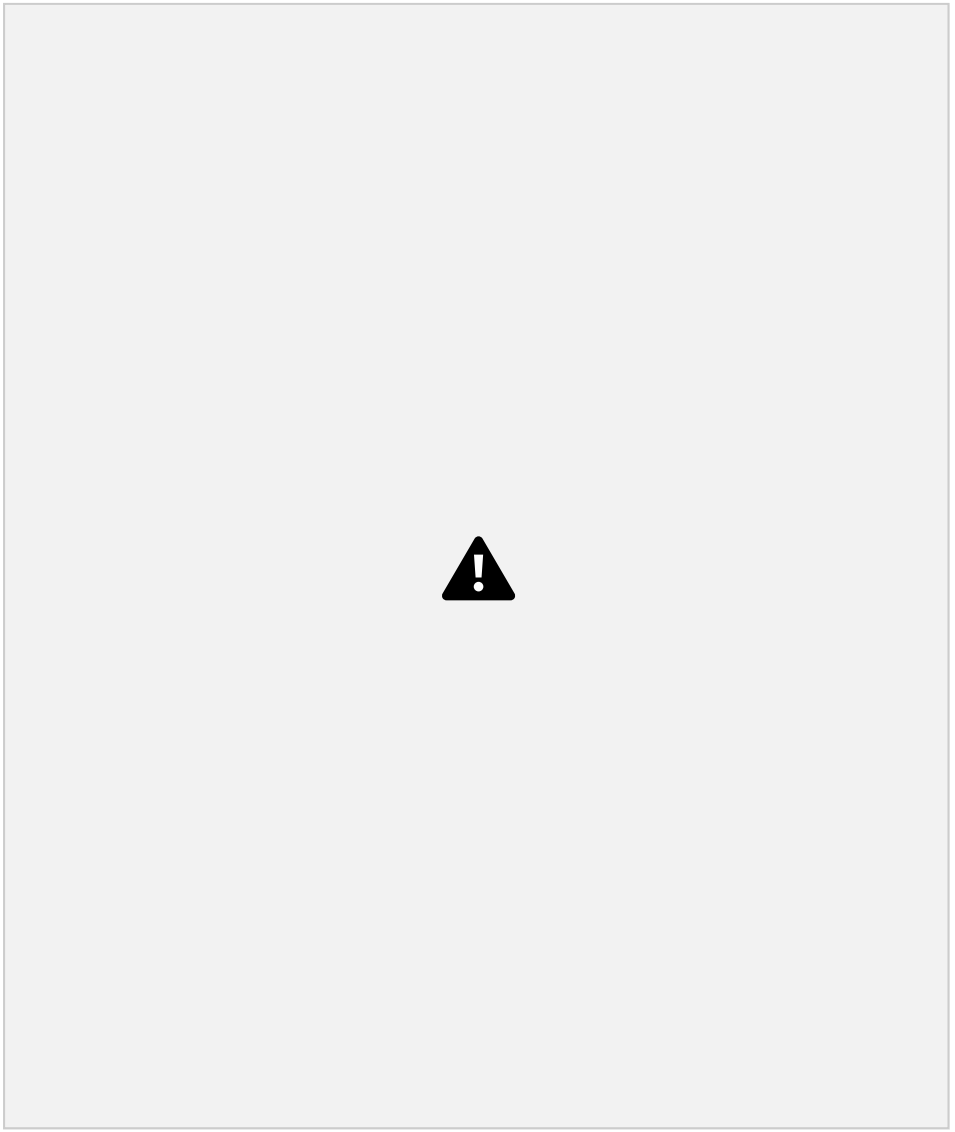
Спрацювання обмеження на кількість та фінальна перевірка цілісності:



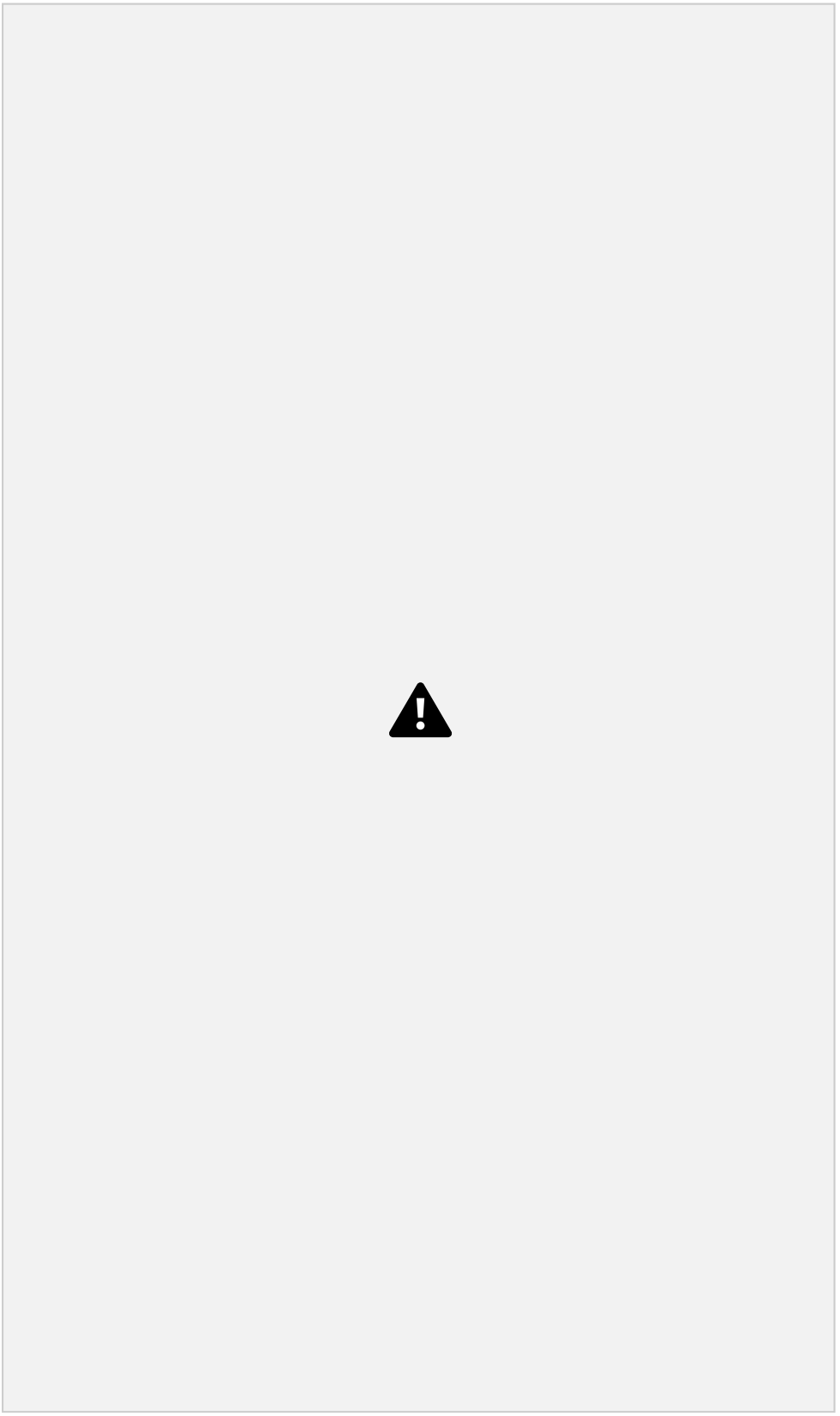
ER-діаграма:

Структуру бази даних було візуалізовано за допомогою вбудованих інструментів СУБД, що підтвердило наявність усіх зв'язків між первинними (PK) та зовнішніми (FK) ключами згідно із заданою архітектурою.









Висновок:

Під час виконання лабораторної роботи було успішно перетворено концептуальну ER-модель інтернет-магазину книг на фактичну реляційну схему в СУБД PostgreSQL. За допомогою інструкцій DDL створено 10 взаємопов'язаних таблиць із чітким визначенням відповідних типів даних, первинних та зовнішніх ключів, а також обмежень NOT NULL, UNIQUE, CHECK та DEFAULT для забезпечення суворої цілісності даних. Заповнення таблиць тестовими даними за допомогою інструкцій DML, виконання аналітичних JOIN-запитів та успішне проведення негативних тестів у середовищі pgAdmin підтвердили правильність побудови схеми, коректність роботи всіх зв'язків та захист бази даних від введення некоректних даних. У результаті роботи було здобуто практичні навички проектування, реалізації та адміністрування реляційних баз даних.